

SchemaForge

An agent that rehearses database migrations, proves what happened, and knows when to stop.

A human states a PostgreSQL change in plain language. SchemaForge inspects the live target, maps dependencies, executes generated SQL inside a rollback-only shadow sandbox, verifies explicit assertions and rollback equivalence, then freezes before production. Applying requires both a signed exact-action token and TrueForge's visible Allow/Deny checkpoint.

6

typed MCP tools

2

isolated services

2

human gates

0

blind prod writes

Repository

github.com/harshaxyZ/schemaforge

EVIDENCE FIRST

HUMAN GATED

Built for the challenge, not around it

The official challenge asks for an agent that reaches a real system, runs generated code safely, and pauses before irreversible action. SchemaForge is the featured “Migration Rehearsal Agent” pattern made concrete.

Requirement	Verifiable implementation	Status
Runs on TrueForge	Current agent spec, focused instructions, remote Streamable HTTP MCP connectors, and sandbox enabled.	YES
Real system	PostgreSQL 16 target reached with a database-enforced read-only role.	YES
Safe generated code	Migration SQL runs on a separate shadow database in one serialized transaction, then always rolls back.	YES
Human checkpoint	MCP destructiveHint, explicit TrueForge tool gate, signed five-minute token, and human -confirm.	YES
Secrets protected	Core has no write credential or signing key; role env files and runtime state are ignored by Git.	YES
Explainable build	Deterministic rules and evidence are implemented in small TypeScript modules; AI use is disclosed.	YES

Why this matters

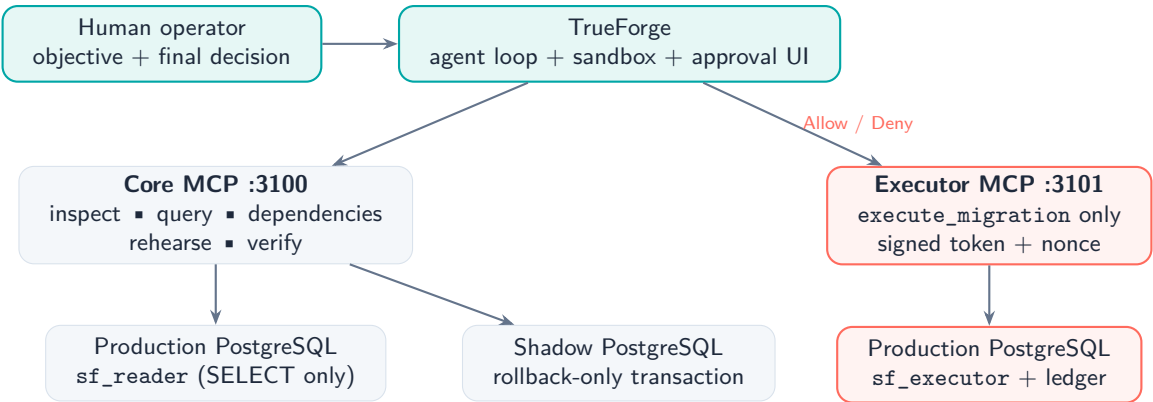
Database migrations combine model uncertainty with high-blast-radius actions. A plausible SQL answer is not proof. SchemaForge converts the risky leap from “looks right” to “run it” into a measured control loop with a deliberate human boundary.

Two demo outcomes

Knows when to stop: adding NOT NULL UNIQUE to seeded emails exposes 14 NULLs and a duplicate group; rehearsal fails and production remains untouched.

Acts when proven safe: an additive phone column rehearses, verifies, rolls back cleanly, pauses twice, and then commits with drift and postcondition checks.

System architecture



The action boundary is enforced in code

Before rehearsal

- Stable schema fingerprint excludes row-count estimates and internal ledger state.
- Dependency catalog covers foreign keys, views, materialized views, functions, triggers, indexes, policies, and sequences.
- Fail-closed SQL allowlist rejects Tier-3, DML, internal-ledger access, ambiguous escapes, unknown, mixed, and non-transactional plans.

Inside the shadow sandbox

- One physical PostgreSQL connection owns BEGIN, SQL, assertions, rollback test, and final ROLLBACK.
- Explicit assertions evaluate values; successful execution alone never passes.
- Returns observed timing, lock snapshot, notices, exact row counts, and three fingerprints.

At human approval

- HMAC covers exact SQL and assertion-set hashes, baseline/post fingerprints, rehearsal ID, action, target, UUID nonce, and expiry.
- The model never receives the signing secret and cannot mint a token.
- TrueForge independently gates the destructive-annotated tool call.

During production execution

- Nonce is atomically claimed, preventing replay and ambiguous retries.
- Advisory lock serializes SchemaForge applies; lock and statement timeouts cap impact.
- Drift check, DDL, approved assertions, post-fingerprint check, and success-ledger update form one transaction.

Defense in depth

READ-ONLY ROLE

SHADOW DB

SQL POLICY

SIGNED TOKEN

NONCE LEDGER

TRUEFORGE GATE

Evidence-driven workflow

#	Stage	Observable output
1	Parse intent	Objective, affected objects, explicit success condition
2	Inspect	Catalog, PostgreSQL metadata, baseline SHA-256
3	Dependencies	Exact blast-radius categories and object definitions
4	Synthesize	Byte-stable forward SQL, honest rollback, transaction class
5	Rehearse	Timings, locks, notices, row deltas, post fingerprint
6	Verify	Named machine assertions plus rollback fingerprint equality
7	Assess	Deterministic APPLY / REVIEW / DO_NOT_APPLY recommendation
8	Freeze	Complete decision packet; no production call in the same turn
9	Execute	New human message, signed token, TrueForge Allow, ledgered result

Measurement discipline: tool-returned values are labeled **[OBSERVED]**; catalog statistics and reasoned projections are **[ESTIMATED]**. A lock snapshot is never presented as an exact lock duration.

Judge demo: failure first, then safe action

01 Prove that stopping is a feature

Prompt: “Make users.email NOT NULL and UNIQUE. Prove it is safe before applying anything.”

The shared deterministic seed contains 500 users, exactly 14 NULL emails, and three rows sharing duplicate@example.com. Inspection and data queries surface the violations. Naive DDL fails in the shadow transaction, the tool returns DO_NOT_APPLY, and SchemaForge freezes without calling the executor.

02 Demonstrate controlled agency

Prompt: “Add an optional phone VARCHAR(32) column. Rehearse it, verify rollback, and prepare a decision packet.”

The agent runs forward SQL, evaluates a catalog assertion, captures the post fingerprint, runs rollback SQL, proves baseline restoration, and presents the packet. The operator signs the exact SQL outside the agent. A new turn calls the destructive tool; TrueForge pauses. On Allow, the executor checks drift and postconditions before commit. Reusing the token proves replay rejection.

Fast local launch

```
$ npm run setup      $ npm run build
$ docker compose up -d
$ npm run start:core  # terminal 1
$ npm run start:executor # terminal 2
$ npx @truefoundry/trueforge@0.2.1
```

What judges can inspect

- Six typed MCP tools with correct annotations
- Separate core/executor credentials and ports
- SQL policy and HMAC verifier
- Durable production nonce/outcome ledger
- Deterministic edge-case data
- Focused TrueForge agent instructions

Deliberate limits

SchemaForge’s atomic executor refuses all DML, concurrent-index and other non-transactional DDL rather than pretending it can roll them back. It also refuses role administration, database/schema/table deletion, procedural escape hatches, internal-ledger access, and SQL outside its canonical fingerprint model. Safe inability is part of the product.

Build story

The first prototype had the right narrative but unsafe mechanics: stdio-only transport, mismatched environment names, pool-level pseudo-transactions, a caller-authored “token,” persistent shadow state, and verification-by-query-success. The v2.1 build replaced each promise with an enforceable boundary and rewrote the public documentation to describe only what the code does.

Team principle

AI assistance: Kiro IDE with Claude Opus 5; architecture remains team-owned and explainable.

Model proposes. Tools measure. Policy constrains.
Human decides.

[View source](#)

Sources: official HackCulture event brief and rules; TrueForge repository and documentation for agent specs, MCP connectors, sandboxing, and human checkpoints. Descriptions are paraphrased for licensing compliance.